

Activity: A Robust, Concurrent Arithmetic Server

Design, Measurement, and Analysis

Course: CSc 113 — Computer Networks and Data Communications

Unit: 2 — Network Programming

Builds on: Activity: *Modify the Socket Server*

Learning Objectives

After completing this activity, you will be able to:

- Implement a stateful, concurrent TCP server with per-connection state.
- Design and implement a simple application-layer protocol on top of TCP.
- Correctly handle TCP message framing — partial reads, pipelined commands, and clients that send data byte-by-byte.
- Make the server robust against malformed input, slow clients, and abrupt disconnects.
- Measure the empirical behavior of your own server (RTT, throughput, concurrency).
- Instrument your server with `recv()` logging to observe how TCP delivers application messages.
- Justify your design decisions in writing, with reference to your own code and logs.

Note on academic integrity and AI tools. Several deliverables in this activity (screenshots from your server, `recv()` logs from your traffic, measurements taken on `hpc1`, design justifications that reference **your** specific code) cannot be produced by an AI assistant on your behalf. You may use AI tools to help you understand concepts or debug, but the empirical artifacts and the written analysis must be your own. Be prepared to defend your submission in a brief oral check.

Background

In the previous activity, *Modify the Socket Server*, you adapted the echo server so it could handle multiple connections concurrently. In this activity you will turn that server into an Arithmetic Server that speaks a small, well-defined protocol — and then you will stress-test it, measure it, and analyze it.

A common mistake when first writing TCP servers is to assume that one `recv()` call yields exactly one message. **It does not.** TCP is a byte stream: a single `recv()` may return a partial command, an entire command, or several commands concatenated. Your server must handle all three cases. Much of this activity is about confronting that fact head-on.

Setup

1. SSH into `hpc1.msuiit.edu.ph` (27.110.175.11).
2. Copy your concurrent server from the previous activity:

```
$ cp modified_socket_echo_server.py arithmetic_server.py
```

3. Edit with `nano -w arithmetic_server.py`. You may also use a personal git repo if you prefer.
4. Run it: `python3 arithmetic_server.py`.
5. From a second SSH session, connect with `telnet localhost <your port>`.

Part 1 — Protocol Definition

Your server must implement the protocol below. Behavior must match exactly — no extra prompts, no missing whitespace, no different wording.

Client requests

Command	Meaning
ADD <N1> <N2>	Add N1 and N2.
SUB <N1> <N2>	Subtract N2 from N1.
MUL <N1> <N2>	Multiply N1 by N2.
DIV <N1> <N2>	Integer-divide N1 by N2.
RND <N>	Return a random integer in [1, N].
HIST	Show up to the last 5 valid operations for this connection only.
HELP [command]	Show all commands, or detailed help for one command.
QUIT	End the session.

Server responses

- OK <result> — on success.
- ERR <message> — on any error.

On connect, the server sends: `OK Welcome to the CSc 113 Arithmetic Server!`

On QUIT, the server sends `OK Bye.` and then closes the connection.

Response framing. All server responses end with a newline (`\n`). Multi-line responses (like `HIST` or `HELP`) consist of multiple `\n`-terminated lines; the final line of the response is the last one shown in Appendix A. Without trailing newlines, telnet renders responses on top of subsequent input and a programmatic client cannot tell when one response ends and the next begins.

Sample client–server transcripts are provided in **Appendix A**. Your output must match those transcripts character-for-character (apart from random values from `RND`).

Part 2 — Robustness Requirements

Your server must survive every situation in this section without crashing, hanging, or leaking the misbehaving client into other clients' sessions.

R1. Per-connection state. Two clients connected at the same time must each have their own independent 5-entry history. One client's HIST must never show the other client's operations.

R2. Message framing. A command ends with a newline (`\n`). The client may send `\r\n` (telnet typically does) — accept both. You must correctly handle all of:

- One `recv()` call returning a *partial* command (e.g. "AD", then later "D 1 2\n").
- One `recv()` call returning *multiple* commands ("ADD 1 2\nSUB 3 4\n").
- A command sent one byte at a time, with arbitrary delays.

R3. Bounded commands. A single command line is at most **256 bytes** including the newline. If a client sends more than that without a newline, respond `ERR Command too long.`, then either resynchronize at the next newline or close the connection — your choice, but document it.

R4. Malformed input. Non-numeric arguments, wrong argument counts, unknown commands, blank lines: each must yield a specific `ERR ...` message. Blank lines should be silently ignored (no response).

R5. Disconnects. If a client closes the socket (or its connection drops) in the middle of sending a command, your server must clean up and keep serving everyone else.

R6. No global state pollution. A buggy or hostile client must not be able to corrupt another client's history, crash the server process, or starve other clients of service.

Part 3 — Empirical Measurement

You will measure your own server's behavior on `hpc1`. **Report exact numbers from your runs.** Vague answers like "it was fast" earn no credit.

M1. Round-trip time. Write a short Python client (≤ 40 lines) that:

1. Opens a connection to your server.
2. Sends 1,000 `ADD 1 2` commands sequentially, waiting for each response before sending the next.
3. Reports the mean and median RTT in milliseconds.

Run it three times. Report all three results and comment on the variation.

M2. Throughput. Modify your client so it sends all 1,000 `ADD` commands in pipelined fashion — i.e. send every command without waiting, then read all 1,000 responses. Report total elapsed time and operations per second. Compare with M1.

M2 should be noticeably faster than M1. If it isn't, investigate why — most likely your server is starving the network (e.g. calling `recv()` with a tiny buffer, sleeping between reads, or flushing inefficiently). The bottleneck is unlikely to be CPU; `ADD` is trivially cheap.

M3. Concurrent clients. Run 5, 10, and 20 instances of your M1 client at the same time. Report mean RTT for each level of concurrency. Does it scale linearly, sub-linearly, or worse? Speculate (one paragraph) on why, with reference to your server's concurrency model (threads? `select`? `asyncio`?).

Part 4 — Observing the TCP Byte Stream (`recv()` Logging)

A common misconception is that one `recv()` call returns exactly one command. In this part you will *prove to yourself* that it does not, by instrumenting your server and observing real reads.

Instrument your server. Wherever your server calls `socket.recv()`, log:

- A timestamp (millisecond precision is fine).
- The client's address.
- The number of bytes returned.
- The raw bytes, shown with `repr()` so that `\r`, `\n`, and partial commands are visible.

Example log line:

```
[12:04:33.812] 127.0.0.1:51422 recv() -> 4 bytes b'ADD '  
[12:04:35.001] 127.0.0.1:51422 recv() -> 6 bytes b'1 2\r\n'
```

Write the log to a file (e.g. `recv.log`) so you can include excerpts in your report.

Capture three scenarios. Run your server and exercise it three different ways, saving each log:

L1. Telnet — observe and explain. Connect with telnet and send `ADD 1 2` followed by Enter. Look at your log and *count* the `recv()` calls that delivered the command. You may see one call (the whole line arrived at once), or you may see several (the bytes arrived in chunks). Both outcomes are correct — they depend on how your particular `telnet` is configured (line mode vs. character mode) and on whether the server negotiated telnet options.

In your report: paste the relevant log excerpt and *explain what you observed* in one or two sentences. The point of L1 is not to produce a particular number of `recv()` calls — it is to confront the fact that you, the server author, do not control how the client sends. You must handle whatever arrives.

L2. Pipelined benchmark client. Run the pipelined client from Part 3 (M2). In your log you should see `recv()` returning **multiple commands at once** — a single read whose bytes contain several complete `ADD 1 2\n` lines. Paste an excerpt and point to one such read.

L3. Drip client. Write a tiny client (≤ 20 lines) that sends `ADD 1 2\n` one byte at a time with `time.sleep(0.3)` between bytes. Confirm in your log that your server reads one byte per call, accumulates them, and still produces the correct `OK 3` response. Paste the excerpt.

Analyze. Answer the following in your report, with reference to specific lines in your log:

1. In L1, how many `recv()` calls did your server make to receive one `ADD 1 2` command? Explain what you observed — does your telnet appear to be in line mode or character mode? How can you tell from the log?
2. In L2, what is the maximum number of complete commands you saw in a single `recv()` return? What does this tell you about the relationship between application messages and TCP reads?
3. In L3, your server received the command one byte at a time but still answered correctly. Point to the lines in `arithmetic_server.py` that made this work, and explain in 2–3 sentences how the receive buffer behaves.

Part 5 — Original Extension (Your Design)

Add **one** new command of your own design. Examples (do **not** use these — invent your own):

- `STATS` — return number of operations served this session and uptime.
- `EVAL <expr>` — evaluate a small arithmetic expression in postfix notation.
- `UPPER <text>` — uppercase a string (a non-arithmetic command — what does this say about your server?).

In your report (½–1 page):

1. State the syntax and semantics of your command.
2. Show a sample successful interaction and one error case.
3. Justify why this command is a reasonable addition to this server. What design principle does it respect or violate?

Part 6 — Design Document (≈ 2 pages)

Answer these questions in prose. **Quote line numbers from your** `arithmetic_server.py` **where relevant.**

1. How does your server know when a client has finished sending one command? Walk through what happens, in your code, when a client sends "ADD 1" and then pauses for 10 seconds before sending " 2\n". Which function in your code holds the partial buffer? Where?
2. What is your concurrency model — threads, select/selectors, asyncio, fork? Why did you choose it? What is one situation in which it would scale poorly?
3. Suppose a malicious client opens a connection and sends one byte every 30 seconds, never a newline. What does your server do? Is that the behavior you want? If you wanted to defend against this, how would you change your code? (You do not have to implement the defense — describe it.)
4. Your protocol uses newline-delimited ASCII. Name one advantage and one disadvantage of this choice compared with a length-prefixed binary protocol.
5. Refer to your `recv()` log in L3. The string "ADD 1 2" arrived across multiple `recv()` calls — one per byte. Why did your server still handle the command correctly? Point to the specific lines in your code that made this work.

Deliverables

Submit a single ZIP archive (`<surname>_arithmetic.zip`) containing:

1. `arithmetic_server.py` — your final server, with comments.
2. `bench_client.py` — the measurement client from Part 3.
3. `drip_client.py` — the byte-at-a-time client from Part 4 (L3).
4. `screenshots.pdf` — labeled screenshots of: all 8 base commands working (ADD, SUB, MUL, DIV, RND, HIST, HELP, QUIT); two simultaneous telnet sessions showing independent HIST (Part 2, R1); at least three robustness scenarios from Part 2 (R2 partial command, R3 oversized command, R4 malformed input); your original extension command in use (Part 5).
5. `logs/` — `recv.log` excerpts from scenarios L1, L2, and L3.

6. report.pdf — design document (Part 6) + measurement report (Part 3) + Part 4 analysis + extension write-up (Part 5).

Submit on MOLE 3.0 under Activity: A Robust, Concurrent Arithmetic Server.

Grading Rubric (100 points)

Component	Points
Protocol correctness — all 8 commands match Appendix A	20
Per-connection state and concurrency (Part 2, R1, R6)	10
Message framing — partial reads & pipelining (Part 2, R2)	15
Robustness — R3, R4, R5	10
Empirical measurement (Part 3)	10
TCP byte-stream analysis via recv() logs (Part 4)	10
Original extension (Part 5)	5
Design document (Part 6)	15
Code quality, comments, readability	5

Note. Deliverables 4 and 5 must clearly show *your* server on *your* port. Generic screenshots or logs that could come from anywhere will be returned for resubmission with no credit on the affected component.

Appendix A — Reference Transcripts

A1. Normal session

```
$ telnet 127.0.0.1 <port>
OK Welcome to the CSc 113 Arithmetic Server!
ADD 1 2
OK 3
SUB 3 4
OK -1
MUL 5 6
OK 30
DIV 40 3
OK 13
DIV 12 0
ERR Division by 0.
RND 100
OK 37
HIST
OK The last valid operations from this session (up to 5) are:
ADD 1 2 -> 3
SUB 3 4 -> -1
MUL 5 6 -> 30
DIV 40 3 -> 13
RND 100 -> 37
QUIT
OK Bye.
```

A2. Invalid argument count

```
MUL 3
ERR Invalid number of arguments to MUL.
```

A3. Unknown operation

```
XYZ 2 1
ERR Unknown operation XYZ.
```

A4. HELP variants

```
HELP
OK The following commands are available:
ADD <N1> <N2> - to add N1 and N2
SUB <N1> <N2> - to subtract N2 from N1
MUL <N1> <N2> - to multiply N1 by N2
DIV <N1> <N2> - to divide N1 by N2
RND <N> - to generate a random number between 1 and N, inclusive
HIST - to show the last 5 valid operations in the session
HELP [command] - to display the syntax and semantics of a specific
command. If no command is specified, it will display all the available
commands and their meanings
QUIT - to end the current session of the arithmetic server

HELP ADD
OK ADD <N1> <N2> - to add N1 and N2

HELP HELP
```

```
OK HELP [command] - to display the syntax and semantics of a specific
command. If no command is specified, it will display all the available
commands and their meanings
```

Appendix B — Hints

- The `socket.recv(bufsize)` call returns whatever bytes are currently available, up to `bufsize`. It is **not** line-oriented. You will need to keep a per-connection buffer of bytes received so far, and parse out complete lines from it (look for `\n`).
- For Part 3, `time.perf_counter()` gives you sub-microsecond timing.
- For Part 4 logging, format bytes with Python's `repr()` so escape sequences are visible: `print(repr(data))` prints `b'ADD 1 2\r\n'` rather than an actual newline.
- If you choose threads for concurrency, beware of shared state. If you choose `asyncio` or selectors, beware of one slow client blocking the loop.

Test your server against itself. Write a buggy client on purpose (sends `\r\n` only; sends one byte per second; sends 10 KB without a newline) and watch how your server reacts.